

Linear containers for prompt/response systems: intuition, with the formal definitions attached

MacBeth
for Neil

2026-08-22

Abstract

This is the readable companion to `containers-over-vec.tex` — but this time with the definitions written out, because you asked for both the intuition *and* enough formal detail to actually do mathematics. Every abstract notion below lands on a concrete prompt/response example *and* is given a precise definition. One sentence runs through all of it, and it is yours: *a response is a vector because a response is uncertain, and uncertainty is what a learning system trains*. The technical note carries the proofs; this note is so you can decide whether it is worth your evening, and it answers, inline, the seven questions you raised in reply. I have flagged throughout what is *proved* and what is *proposal*.

1 The setup, with definitions

Model a prompt as the simplest possible container: one request, and a space of possible replies. The only real design choice is the word *space*, and it is worth defending before we formalise. Why a vector space A and not just a set of allowed answers? Because a reply is not one token. It is a distribution — a superposition over a basis of possible answers, carrying the model’s uncertainty. The basis is the vocabulary of possible replies; the coefficients are the confidence; and the basis in which that reply space is expressed is exactly what a learning system *learns*. So: responses = uncertainty = the basis of learning. That single sentence is the reason **Vec**, the category of k -vector spaces over a field k , is the right home rather than **Set**. Keep it in view; every section below is a consequence of it.

Now the formal object. It is the ordinary container definition of Abbott–Altenkirch–Ghani, with the base of the position fibres changed from **Set**^{op} to **Vec**^{op}.

Definition (linear container). A *linear container* is an object $(S, (P_s)_{s \in S})$ of **Fam**(**Vec**^{op}): a set S of *shapes* (the prompt addresses), together with, for each $s \in S$, a k -vector space P_s (the *response space* at that shape). A morphism $(S, (P_s)) \rightarrow (T, (Q_t))$ is a pair $(f, (\varphi_s))$ where $f: S \rightarrow T$ is a function and, for each s , $\varphi_s: Q_{f(s)} \rightarrow P_s$ is a linear map — contravariant on positions, exactly as for **Set** containers.

Definition (extension). The *extension* of $(S, (P_s))$ is the endofunctor $\llbracket S, P \rrbracket: \mathbf{Vec} \rightarrow \mathbf{Vec}$,

$$\llbracket S, P \rrbracket W = \bigoplus_{s \in S} [P_s, W], \quad [P_s, W] := \mathbf{Vec}(P_s, W) = \{\text{linear maps } P_s \rightarrow W\}.$$

Choosing a basis of P_s (dimension n_s) gives $[P_s, W] \cong W^{n_s}$, so $\llbracket S, P \rrbracket W \cong \bigoplus_s W^{n_s}$: a direct sum of powers of W .

The single prompt is then $A = (\{*\}, A)$, one shape over the reply space A , with extension $\llbracket A \rrbracket W = [A, W]$: a point of it is a linear map $A \rightarrow W$, i.e. a way of reading the reply space into an ambient answer space W . Notice already the two structural ingredients that the whole note will turn on: an *internal hom* $[P_s, -]$ on the inside, and a *biproduct* $\bigoplus_s$ on the outside. Section 2 says exactly which structures of $\mathbf{Vec}$ these are, and answers your question (B) about “closed with respect to what”.

(A) Should prompts also be a vector space, or stay a set? This is a genuine design axis, and worth naming as one. In the definition above the *prompt* (the shape $s \in S$) is a plain element of a set, while only the *response* P_s is a vector space. That is the recommended starting theory: $\mathbf{Fam}(\mathbf{Vec}^{\text{op}})$, a *set* of prompt-addresses each carrying a trainable response space. The prompt is the address/shape; the response is the trainable content. The alternative — committing to a “better”, fully linearised theory in which the *prompts themselves* form a vector space and can be *superposed* (a linear combination of prompt-addresses is again a prompt) — is not a different axiom bolted on; it is precisely the move *up* to the $\mathbf{Vec}$ -matrix / algebroid layer of Section 3, where prompts become vertices $a, b, \dots$ that one may take linear combinations of. Here is the striking point worth flagging: *attention already superposes prompts*. A query is a vector, and softmax over keys returns a weighted combination of key-addresses — a superposition of prompt-addresses, not a choice of one. So “commit fully to vector-space prompts” is the same act as “move up to the $\mathbf{Vec}$ -matrix/enriched layer”; adopt it exactly when you want to superpose prompts, and stay in $\mathbf{Fam}(\mathbf{Vec}^{\text{op}})$ when the address is a discrete handle.

2 The structure of $\mathbf{Vec}$ we use, and what it lacks

We lean on three facts about $\mathbf{Vec}$ and one fact it is *missing*. Each gets a one-line intuition on the prompt example and a precise statement.

The zero object.

The empty reply space and the terminal (“full”) reply space are the *same* object 0 : it is at once initial and terminal ($\exists! V \rightarrow 0$ and $\exists! 0 \rightarrow V$). In $\mathbf{Set}$ the empty set and the one-point set are wildly different; in $\mathbf{Vec}$ they collapse. Harmless-looking, but it is the seed of everything — in particular the $\mathbf{Set}$ slogan “ $F(1) = \text{shapes}$ ” becomes $\llbracket S, P \rrbracket(0) = 0$ and sees nothing.

The biproduct $A \oplus B$ — the cleanest hook.

For finitely many spaces the coproduct and the product *coincide*: $A \oplus B$ carries both the injections $A, B \hookrightarrow A \oplus B$ (making it the coproduct) and the projections $A \oplus B \twoheadrightarrow A, B$ (making it the product), compatibly. So $A \oplus B$ is at once “I can answer *either* prompt A or prompt B ” (coproduct) and “I am ready for *both* A and B ” (product). Same object. In $\mathbf{Set}$ these come apart — a tagged union $A \sqcup B$ versus a pair $A \times B$ — and that gap is the whole difference between the two worlds.

(B) $\mathbf{Vec}$ is closed — with respect to $\otimes$, not $\oplus$.

You asked precisely this. The internal hom $[A, X] = \mathbf{Vec}(A, X)$ is again a vector space because $\mathbf{Vec}$ is a *closed* monoidal category, and the structure it is closed *for* is the tensor product $\otimes$,

not the biproduct. The defining adjunction is

$$\mathrm{Hom}(B \otimes A, X) \cong \mathrm{Hom}(B, [A, X]), \quad \text{i.e.} \quad (- \otimes A) \dashv [A, -].$$

The internal hom is the right adjoint to tensoring with A . The biproduct $\oplus$ is the (co)product; it is *not* a closed structure (there is no adjunction $(- \oplus A) \dashv [\dots]$ of this shape). So the extension $\llbracket S, P \rrbracket W = \bigoplus_s [P_s, W]$ genuinely *mixes two structures*: the $\otimes$ -closed internal hom on the *inside* (the response fibre), and the biproduct $\oplus$ on the *outside* (the sum over shapes). That is the crisp answer, and it is worth carrying: the two composition operations of Section 3, $\otimes$ and $\bigoplus_b$, are exactly these two structures pulled apart.

(C) What *Vec* lacks: extensivity.

You proposed the exact definition, and it is the one I mean. *Set* is *extensive*: the canonical functor

$$\mathrm{Set}/(S + R) \xrightarrow{\sim} \mathrm{Set}/S \times \mathrm{Set}/R,$$

built by pullback along the two coproduct injections, is an *equivalence* (Carboni–Lack–Walters 1993). Concretely: an object over a disjoint union *is* the same thing as a pair of objects, one over each summand — you can always recover which branch you are in. My earlier gloss, “you can always tell which branch you’re in,” was the loose paraphrase of exactly this equivalence. Now the *Vec* counterexample, which is the whole story in one line: the analogous

$$\mathrm{Vec}/(A \oplus B) \longrightarrow \mathrm{Vec}/A \times \mathrm{Vec}/B$$

is **not** an equivalence. Take the diagonal subspace $\Delta = \{(a, a)\} \subseteq A \oplus A$: its two projections to the summands are both the identity on A , so the pair (id, id) on the right does not remember that we picked the diagonal rather than all of $A \oplus A$. The functor is not full (nor essentially injective): the diagonal is invisible to its projections. That single failure of one equivalence is the entire content below — shapes go invisible, and the extension stops being full.

The punchline. That one missing equivalence is the whole story. Because $\mathrm{Vec}/(A \oplus B)$ is not $\mathrm{Vec}/A \times \mathrm{Vec}/B$, the shapes of a container go *invisible* (finitely many prompts with reply spaces of total dimension N become indistinguishable from N blank copies of the identity functor — the “biproduct collapse”), and the map that turns a container into its functor stops being *full* (there are more natural transformations than container morphisms, because a natural transformation may *mix* branches while a container morphism must *pick* one). Formally the two hom formulas differ in exactly one symbol,

$$\underbrace{\prod_s \prod_t \mathrm{Vec}(Q_t, P_s)}_{\text{container morphisms}} \hookrightarrow \underbrace{\prod_s \bigoplus_t \mathrm{Vec}(Q_t, P_s)}_{\text{natural transformations}}, \quad \prod \subsetneq \bigoplus,$$

the inclusion of the disjoint union into the biproduct. Loss of shapes, loss of fullness: two faces of the one missing partition. Proofs are in `containers-over-vec.tex` (Thms. 5.2, 6.4); here I only want you to see that it is a single fact wearing two masks.

3 Composing responses: monoid in *Vec*, then *Vec*-enriched category

Now: what does it mean to *compose* responses? There are two levels, and keeping them apart resolves the confusion you flagged in (F).

(D) Within one prompt — a monoid in $(\mathbf{Vec}, \otimes)$. Stay with a single prompt A . Suppose you can take a response, run it, and feed its result back as another response of the *same* prompt, then read off one combined response. That is a map $\mu: A \otimes A \rightarrow A$ combining two replies into one, associative, with a unit $\eta: k \rightarrow A$ (the empty response). This is exactly a k -algebra — and, in your preferred and more precise terminology, it is a *monoid object in the monoidal category* $(\mathbf{Vec}, \otimes, k)$. (A plain monoid is a monoid in $(\mathbf{Set}, \times)$; the k -algebra is the very same notion, taken internal to $\mathbf{Vec}$ with $\otimes$ for $\times$.) So within a single prompt the entire composition structure is: a monoid on its reply space. Nothing more is available, because there is only one prompt and nothing to route through.

(E) Across prompts — a $\mathbf{Vec}$ -enriched category (algebroid). Now allow several prompts and responses that carry *one* prompt into *another*: a response has a source prompt a and a target prompt b . Over $\mathbf{Set}$ this “responses carry object to object, and compose” is precisely the theorem *directed containers = small categories* (Ahman–Chapman–Uustalu). The $\mathbf{Vec}$ version is an *algebroid*, and here again your reading is the right formalisation: an algebroid is a *category enriched in* $(\mathbf{Vec}, \otimes)$ — a category whose hom-sets have been upgraded to hom-spaces $\mathrm{hom}(a, b) = P(a, b) \in \mathbf{Vec}$, with k -bilinear composition. (This is Mitchell’s “ring with several objects”: a one-object $\mathbf{Vec}$ -category is a single k -algebra, i.e. the monoid of (D).) Prompts are the objects; between prompts a and b sits a whole reply space $P(a, b)$ of responses carrying a to b .

The right way to package this is a *Vec-matrix*: prompts on rows and columns, entry $P(a, b)$ the reply space from a to b . Composition is honest matrix multiplication,

$$(P \odot Q)(a, c) = \bigoplus_b P(a, b) \otimes Q(b, c) \quad (\text{“sum over every intermediate prompt } b\text{”}),$$

with unit the matrix $I(a, b) = k$ if $a = b$, else 0. The *monoids* of $(\mathbf{Mat}_S(\mathbf{Vec}), \odot, I)$ are exactly algebroids: $\mu: P \odot P \rightarrow P$ is the family of composition maps $\bigoplus_b P(a, b) \otimes P(b, c) \rightarrow P(a, c)$, and the unit picks out the identity response at each prompt.

(F) How can it be both a $\triangleleft$ -comonoid and an algebroid? Here is the resolution, and it turns on your own insight. A directed container is a *comonoid in* $(\mathbf{Cont}, \triangleleft)$, where $\triangleleft$ is substitution of containers. Over $\mathbf{Vec}$ at a *single* index the $\triangleleft$ -comonoid *degenerates*: the counit forces the shape-comultiplication onto the diagonal $s \mapsto (s, s)$, so distinct prompts never interact and you recover only a *family of one-object monoids-in-Vec* (one k -algebra per prompt, no cross-prompt composition). This is proved: $\mathbf{Comon}_{\triangleleft}(\mathbf{Fam}(\mathbf{Vec}_{\mathrm{fd}}^{\mathrm{op}})) \cong \mathbf{Fam}(\mathbf{Alg}_k^{\mathrm{op}})$. The genuine *multi-object* algebroid needs the $\mathbf{Vec}$ -matrix composition $(P \odot Q)(a, c) = \bigoplus_b P(a, b) \otimes Q(b, c)$ instead. And — this is your observation, and it is the key — that matrix product *is profunctor composition*. A profunctor composes by a coend $\int^b P(a, b) \otimes Q(b, c)$; for a *discrete* finite index category the coend has no nontrivial identifications and becomes the finite biproduct $\bigoplus_b$. So:

$\mathbf{Vec}$ -matrices = profunctors between discrete $\mathbf{Vec}$ -categories;

algebroid = a monad in the bicategory of $\mathbf{Vec}$ -profunctors = a $\mathbf{Vec}$ -category.

That is what dissolves the paradox. The single-index $\triangleleft$ is the *diagonal* degeneration — one hom-space per shape, arrows only from a prompt to itself; the matrix $\triangleleft$ (i.e. $\odot$) is the *real thing* — one hom-space per *ordered pair*, arrows between distinct prompts, composed by the coend. Both are “ $\triangleleft$ -comonoids”; they differ in whether the coproduct sits on the shape index

(degenerate) or on the composition index (genuine). The repair is exactly to move the $\oplus$ from the shape index to the composition index. (All of this is Section 7–8 of the technical note; the monoid-in-a-matrix-bicategory = enriched category fact is Bénabou; profunctor composition by coend is standard.)

4 The wirings: a proposal

This is the part I actually want your eye on, and it is where question (G) — the *use* — becomes the headline. You put it exactly: *the core of machine learning is a matrix of weights, and these weights are to be learned, so they come from not a set but something that can be tweaked*. That is precisely the **Vec**-matrix $P(a, b)$ of Section 3: it *is* the trainable weight matrix between prompt-states. Composing workflows is matrix multiply of **Vec**-matrices; learning is tweaking the vectors inside the entries. Responses are vector spaces because a response is uncertain (a superposition over a basis), and that uncertainty is exactly what gets trained. This is why **Vec**, not **Set**, is the home: the things being wired are trainable vectors, not inert tokens.

Read that way, containers over **Vec** carry four monoidal products — $\oplus, \times, \otimes, \triangleleft$ — but in the finite linear setting the biproduct makes two of them *coincide*, and that coincidence is itself the cleanest thing to say. So there are *three* genuinely distinct ways to wire two systems together.

$\oplus = \times$ — **menu / both (one wiring, by the biproduct collapse)**.

Offer prompt A or prompt B . Over **Set** the menu (coproduct $A \sqcup B$) and the pair (product $A \times B$) are different constructions. Over **Vec** they are the *same* object, the biproduct $A \oplus B$ (Section 2): building the menu *is* being ready for both at once. So the two combinators a workflow language would ordinarily keep apart — “choose a branch” and “hold both branches” — collapse into one. This is not a defect to apologise for; it is the biproduct collapse, now seen at the level of workflows. There is exactly one “combine by choice/product” operation, and it is $\oplus$.

$\otimes$ — **parallel / entangling**.

Run two prompt/response systems side by side on independent inputs; the joint reply space is the tensor $A \otimes B$. Genuinely distinct from $\oplus$: the tensor *entangles* the two reply spaces (a general element of $A \otimes B$ is not a sum of a pure- A and a pure- B part) where the biproduct keeps them additively separate.

$\triangleleft$ — **pipeline**.

Feed one system’s response in as the next system’s prompt. This is composition down a chain, and its bookkeeping is exactly the matrix product $\odot$ of Section 3: route the output prompt through to the next stage, $\bigoplus_b P(a, b) \otimes Q(b, c)$.

The **claim** is a clean slogan: *the algebra of wiring is the algebra of workflows*. Three categorical combinators — choose/hold, run-parallel, pipe — as the three connectives of a workflow language, with **Vec** the right home because the things being wired are trainable weight-matrices (this is your (G): learning = tweaking the vectors).

I want to be honest about altitude. This section is a *proposal*, a research direction, not a theorem. In particular two glosses are frankly speculative and I flag them as such: (i) the operational reading of $\triangleleft$ as “feed one system’s response in as the next system’s prompt” is a suggestive interpretation of the matrix product, not a proved semantics; and (ii) the hope that

a workflow assembled this way stays *differentiable in its responses*, so that learning acts on the whole wiring and not only a single response, is exactly the payoff I would want from working over **Vec** — and exactly what remains to be established. That is the “find a use” thrust, and it is where I would go next.

Comparison with existing work (an honest ledger). This framing has neighbours, and I would rather name them than be caught restating them.

- **Vertechi, “Neural network layers as parametric spans.”** A neural-network layer is presented as a parametric span, computing one indexed sum $y[t(p)] += x[s(p)] \cdot w[\pi(p)]$. Structurally this is a container/polynomial *shape* (s, t, π) with a *linear action on positions* — an existing precedent for “Vec-container = NN layer.” Notably it does not cite Abbott/Altenkirch/Ghani or Ahman–Uustalu, so the container reading is available to contribute rather than already claimed.
- **Bradley, “Category Theory Inspired by LLMs.”** A category of linguistic expressions enriched over $[0, 1]$, with enriched copresheaves modelling $\text{text} \rightarrow \text{semantics}$ — a companion categorical-semantics-of-LLMs framework, enriched over a different base than ours but aimed at the same object of study.
- **Honesty about novelty.** The container framing is a genuine contribution *only* if it unifies or sharpens these — e.g. by placing Vertechi’s parametric spans and the **Vec**-matrix algebroid in one composition calculus — and not if it merely restates them in new notation. I believe there is a unification here; I have not yet proved it is more than a translation.
- **Hedges, “Autodiff through function types” (julesh.com, 2026).** This is the universality companion, and it is striking. Hedges shows additive lenses are *cartesian closed* precisely *because* commutative monoids have biproducts ($\coprod = \oplus$) — the *identical* biproduct-collapse lemma I proved, surfacing independently under autodiff/gradient types. So “biproduct $\Rightarrow$ either = both” is not a **Vec** artifact: it is structural wherever an additive structure sits under a container-like formalism. That the same lemma governs my collapse and Hedges’s closed structure is, to me, the strongest sign the phenomenon is real.

5 Status, honestly

Proved (all in `containers-over-vec.tex`):

- *Biproduct collapse.* Finitely many prompts with total reply-dimension N are indistinguishable, as a functor, from N blank copies of the identity ($\llbracket S, P \rrbracket \cong \text{Id}^N$); the shape set is not recoverable.
- *The extensivity crux.* The hom formula $\text{Nat}(\llbracket S, P \rrbracket, \llbracket T, Q \rrbracket) \cong \prod_s \bigoplus_t \text{Vec}(Q_t, P_s)$ shows the container-to-functor map is faithful but *not full*, and pinpoints the failure in the one symbol $\coprod \subsetneq \bigoplus$ — the failure of the extensivity equivalence of (C).
- *$\triangleleft$ -comonoid = family of k -algebras.* The single-index composition gives exactly one monoid-in-**Vec** per prompt (self-loops only), no cross-talk: $\text{Comon}_{\triangleleft}(\text{Fam}(\text{Vec}_{\text{fd}}^{\text{op}})) \cong \text{Fam}(\text{Alg}_k^{\text{op}})$.
- *The correct home for algebroids.* Linear directed containers live in the matrix bicategory

$\mathbf{Mat}(\mathbf{Vec})$; its (co)monoids are genuine algebroids (= $\mathbf{Vec}$ -enriched categories, Bénabou/Mitchell; $\odot$ = profunctor composition), and the single-index theory is its diagonal degeneration.

Proposal, not yet proved:

- The three-wirings workflow calculus of Section 4 — that choose/hold, run-parallel, and pipe form a usable algebra of prompt/response workflows, that the pipeline reading of $\triangleleft$ is the right semantics, and that $\mathbf{Vec}$ genuinely buys *trainability at the workflow level* (not only at the single response). This is the direction, not a result. (One honest side-benefit is already banked: over $\mathbf{Vec}$, choice and product are the *same* wiring $\oplus = \times$, so the calculus has three connectives, not four.)

That is the honest scorecard. The mathematics that is solid is elementary once you accept the base change; the excitement is that the one fact which breaks the representation theorem (non-extensivity) is exactly the fact an agent that must answer whatever it is asked would *want* — a reply space that is at once “either” and “both”, and a weight-matrix you can tweak. Whether that reading pays off as a real, differentiable workflow calculus is the open bet, and it is the one I would like your steer on.